

# CP5 — Checkpoint

## Anonimização de dados para exportação externa (LGPD)

FIAP Bank · RPA aplicado com IA · Tecnólogo em Inteligência Artificial

**Modalidade: individual · Valor: nota do checkpoint · Prazo: domingo, 04 de outubro de 2026 as 23h59.**

Ferramentas: apenas free-tier. Crie com antecedência as chaves de API do Gemini (aistudio.google.com) e do Groq (console.groq.com).

Entrega: repositório Git (GitHub). Não serão aceitos arquivos soltos ou .zip — ver seção 6.

### 1. Contexto

O robô do FIAP Bank já recebe manifestações de clientes, classifica a urgência (Aula 10), roda em escala com resiliência a falhas (Aula 11) e extrai campos estruturados validados por schema — tipo de manifestação, sentimento, valor citado, urgência e um resumo gerado por IA (Aula 12). Tudo isso fica persistido em SQLite.

Agora a área de Compliance quer contratar uma consultoria externa de analytics para cruzar o histórico de manifestações e identificar padrões de insatisfação recorrentes. Antes de qualquer dado sair do banco para um terceiro, a LGPD exige que informação pessoal identificável (PII) seja removida ou mascarada — e essa remoção não pode depender de enviar o texto para mais um serviço externo (Aula 13), o que apenas trocaria um problema de exposição por outro.

O CP5 pede a última etapa da esteira: antes da exportação, o robô precisa gerar uma versão anonimizada de cada manifestação, sem perder o dado original internamente (ele continua necessário para auditoria e para o próprio atendimento).

### 2. PII considerada

Para efeito desta atividade, é considerada PII qualquer um dos seguintes elementos, sempre que aparecerem no texto ou no resumo gerado pela IA:

- Nome de pessoa — cliente, atendente ou qualquer terceiro citado.
- CPF, formatado (000.000.000-00) ou não (000000000000).
- RG, formatado ou não.
- Número de conta e/ou agência bancária.
- Endereço de e-mail.

*Atenção: o nome de um atendente ou gerente citado dentro do texto também é PII e deve ser mascarado — a obrigação não se limita ao nome do cliente que abriu a manifestação.*

### 3. Workflow esperado

- a) Ler as manifestações de manifestacoes\_clientes\_cp5.csv.
- b) Extrair os campos estruturados de cada manifestação via LLM com response\_schema/Pydantic (Aula 12).
- c) Aplicar a pilha de resiliência da Aula 11: retry com backoff, throttling, fallback Gemini → Groq, cache e checagem de idempotência antes de reprocessar.
- d) Persistir texto, resumo e campos extraídos no SQLite.
- e) Gerar texto\_anonimizado e resumo\_anonimizado com anonimizar\_texto(), sem sobrescrever as colunas originais.

- f) Persistir as versões anonimizadas em colunas próprias.
- g) Produzir um relatório de auditoria comparando amostras antes/depois.

## 4. Requisitos técnicos obrigatórios

### a) Resiliência (peso 25%)

Reaproveite a pilha da Aula 11: retry com backoff exponencial, throttling proativo, fallback Gemini → Groq e cache. A checagem de idempotência deve consultar o banco antes de reprocessar uma manifestação já persistida.

#### **Demonstração obrigatória do fallback:**

force a falha do provedor principal para parte do lote (por exemplo, invalidando temporariamente a chave do Gemini ou simulando um erro) e comprove, com uma consulta SQL ao próprio banco gerado, que a coluna provedor contém uma mistura real de valores 'gemini' e 'groq'. Sem essa evidência no banco, o critério de resiliência não pode ser pontuado integralmente — não basta o código ter um bloco de fallback que nunca foi de fato exercitado.

### b) Anonimização (peso 25%)

Entregue a função em um arquivo separado chamado exatamente `anonimizar_texto.py`, com esta assinatura fixa:

```
def anonimizar_texto(texto: str) -> str:
```

A assinatura fixa existe porque esse arquivo será testado de forma automatizada contra um conjunto de manifestações que vocês não viram. Por isso a função precisa ser genérica — reconhecer padrões de PII, não decorar os exemplos do CSV entregue.

O spaCy (Aula 13) reconhece nomes de pessoa via NER, mas não foi treinado para reconhecer CPF, RG, número de conta/agência ou e-mail: esses são padrões estruturais, não entidades nomeadas. Uma solução que só repete a Aula 13 (NER de nomes) cobre uma fração pequena da PII real definida na seção 2 — tratem nomes e identificadores estruturados como dois problemas complementares, ambos obrigatórios.

Aplique `anonimizar_texto()` tanto no campo texto quanto no resumo gerado pela IA — o resumo pode reintroduzir um nome que o texto original também continha.

### c) Extração e validação (peso 15%)

Reaproveite o Pydantic model e o `response_schema` da Aula 12. Trate explicitamente o caso em que `resposta.parsed` vier None (resposta truncada ou fora do schema), direcionando para revisão humana em vez de quebrar o pipeline.

### d) Auditoria (peso 15%)

O dataset público não vem com gabarito de PII. Vocês mesmos precisam rotular manualmente pelo menos 15 das 40 manifestações — decidir, lendo cada texto, quais PII existem e de que tipo — e então escrever uma célula/função que calcule, a partir dessa rotulagem própria, o recall (percentual de PII que a função realmente removeu) e a precisão (percentual de texto não-PII que permaneceu intacto).

Este critério não está testando se a função é boa — isso já é medido pelo critério de Anonimização, contra um conjunto que vocês nunca viram. Está testando se vocês sabem construir uma avaliação correta para o que construíram: rotular com critério, medir de verdade, e reconhecer os próprios erros, em vez de eyeballar alguns exemplos e escrever uma impressão geral.

Entreguem um arquivo `auditoria_resultado.json` na raiz do repositório, com este formato exato:

```
{
  "recall_estimado": 0.90,
  "precisao_estimado": 0.99,
  "casos_de_erro": [
```

```
{
  "id": "3", "tipo_erro": "falso_negativo", "descricao": "..."},
  "id": "17", "tipo_erro": "falso_positivo", "descricao": "..."}
}
```

casos\_de\_erro precisa ter pelo menos 2 entradas reais — encontradas rodando a função de vocês, não hipotéticas — cada uma com uma descrição específica do que aconteceu naquele id.

**Este arquivo é cruzado automaticamente com o resultado real.**

Na correção, recall estimado e precisao estimado são comparados com o desempenho de fato da função de vocês sobre o mesmo conjunto público, calculado pelo professor. Uma discrepância grande indica rotulagem apressada. Um relato muito próximo de 100%/100% também é tratado como implausível neste dataset — nem a solução de referência do professor chega lá, porque parte dos erros vem de uma limitação real do spaCy (a mesma discutida na Aula 13) que rotulagem cuidadosa deveria revelar.

### e) Persistência (peso 10%)

Adicione as colunas necessárias (texto\_anonimizado, resumo\_anonimizado, provedor) usando queries parametrizadas. Rodar o pipeline duas vezes sobre o mesmo CSV não deve duplicar linhas nem gastar cota de API duas vezes para a mesma manifestação.

### f) Governança (peso 10%)

Em uma célula markdown, explique por que a anonimização é feita localmente com spaCy em vez de ser delegada a mais uma chamada de LLM (retomando os três motivos discutidos na Aula 13), e por que o dado original, não anonimizado, ainda precisa ser preservado dentro do banco do FIAP Bank mesmo depois de existir uma versão anonimizada para exportação.

A organização do repositório também entra neste critério: histórico de commits com mais de uma entrega (não um único commit final), README explicando como rodar o projeto, e nenhuma chave de API commitada — ver seção 6.

## 5. Critérios de avaliação

| Critério             | Peso | O que é avaliado                                                                                                                                                                   |
|----------------------|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Resiliência          | 25%  | Retry/backoff, throttling, fallback Gemini → Groq com evidência real de disparo (mistura de provedores no banco), cache, idempotência.                                             |
| Anonimização         | 25%  | Função anonimizar_texto() tratando nomes (NER) e identificadores estruturados (CPF, RG, conta/agência, e-mail). Testada contra um conjunto de manifestações não vistas pelo aluno. |
| Extração e validação | 15%  | Uso correto de response_schema / Pydantic (Aula 12), tratamento do caso em que a resposta não pode ser interpretada (parsed is None).                                              |
| Auditoria            | 15%  | Rotulagem manual própria de PII em parte do conjunto público, recall/precisão calculados a partir dela e cruzados com o valor real, mínimo de 2 casos de erro documentados.        |
| Persistência         | 10%  | Colunas adicionais no SQLite, queries parametrizadas, idempotência verificável em uma nova execução.                                                                               |

| Critério   | Peso | O que é avaliado                                                                                                                                      |
|------------|------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Governança | 10%  | Justificativa escrita de por que a anonimização é local (spaCy) e não via LLM externo, e por que o dado original precisa ser preservado internamente. |

## 6. Entregáveis

A entrega é feita por link de um repositório Git (GitHub), não por upload de arquivo. Crie um repositório público (ou privado com o professor adicionado como colaborador) contendo:

- Notebook (.ipynb) com o pipeline completo, do CSV até a persistência das versões anonimizadas.
- Arquivo anonimizar\_texto.py na raiz do repositório, com a assinatura fixa indicada na seção 4b — é ele que será baixado e testado de forma automatizada contra o conjunto oculto.
- Banco .db gerado pela execução, incluindo a demonstração de fallback forçado (ou instruções claras no README de como reproduzi-lo, se o arquivo for grande demais para o Git).
- Arquivo auditoria\_resultado.json na raiz do repositório, no formato exato da seção 4d.
- Relatório de auditoria em prosa (pode estar em células markdown dentro do próprio notebook), comentando os mesmos casos que aparecem no JSON.
- README.md curto: como instalar as dependências e rodar o notebook do zero.
- Arquivo .gitignore cobrindo qualquer arquivo de configuração local com chave de API.

### Nunca commitem uma chave de API.

Gemini e Groq são lidos via `userdata.get()` no Colab (Secrets) ou variável de ambiente local — nunca escritos direto no código. Antes do commit final, confirmam o histórico do repositório: uma chave commitada mesmo que depois removida continua visível no histórico do Git. Se isso acontecer, revoguem a chave exposta em [aistudio.google.com](https://aistudio.google.com/) / [console.groq.com](https://console.groq.com/) e gerem uma nova.

## 7. Observações

- A avaliação recai sobre a arquitetura, o processo e a evidência produzida — não sobre acertar exatamente cada classificação individual do LLM, que pode variar entre execuções.
- Usem apenas ferramentas gratuitas. Gerem as chaves do Gemini e do Groq com antecedência para não perder tempo de prova nisso.
- A nota de Anonimização inclui um teste automatizado contra manifestações que vocês não viram — uma função que apenas reproduz os exemplos do CSV entregue, sem generalizar, tende a ir mal nesse teste.
- Entrega individual, por link de repositório Git. Envie o link até o prazo — commits feitos após o horário de entrega não são considerados na correção.
- Em caso de dúvida sobre prazo ou formato de entrega, confirmem com o professor em sala.